Software Requirements Specification

for
Civic Issue Resolution System
(ResolIssue)

Document Detail	Value
Project	Civic Issue Resolution System (ResolIssue)
Document Version	1.0
Status	Final Draft
Standard Followed	IEEE 830 style structure

Live Demo: resolissue.vercel.app

Table of Contents

1. Introduction 3

1.1 Purpose . . . 3

1.2 Scope . . . 3

1.3 Definitions, Acronyms and Abbreviations . . . 3

1.4 References . . . 3

1.5 Document Overview . . . 4

2. Overall Description 4

2.1 Product Perspective . . . 4

2.2 Product Functions . . . 4

2.3 User Classes and Characteristics . . . 4

2.4 Operating Environment . . . 4

2.5 Design and Implementation Constraints . . . 4

2.6 Assumptions and Dependencies . . . 5

3. Functional Requirements 5

4. External Interface Requirements 6

4.1 User Interfaces . . . 6

4.2 Hardware Interfaces . . . 6

4.3 Software Interfaces . . . 6

4.3.1 Communication Interfaces . . . 6

5. Non-Functional Requirements 6

6. System Architecture 7

6.1 Architecture Overview . . . 7

6.2 Key Components . . . 7

6.3 Technology Stack . . . 8

7. Data Requirements 8

8. Use Cases and Issue Lifecycle 8

8.1 Use Case Summary . . . 8

8.2 Issue Status Lifecycle . . . 9

9. Assumptions, Risks and Future Enhancements 9

9.1 Risks . . . 9

9.2 Future Enhancements . . . 9

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Civic Issue Resolution System (ResolIssue). It describes what the system does, who uses it, the functional and non-functional requirements, and the constraints under which it is built. It is intended for developers, testers, project guides, and anyone evaluating the system.

1.2 Scope

ResolIssue is a web-based, community-driven platform where citizens report civic issues (such as damaged roads, garbage, water leakage or broken street lights), the community supports important issues through likes and comments, and administrators track and resolve them. The platform gives transparency and accountability by showing every issue's status from report to resolution.

In scope:

- User sign up, login and secure sessions
- Reporting issues with title, description, images and precise location
- Community engagement through likes (upvotes) and comments
- Issue lifecycle tracking: Pending, In-Progress, Resolved
- Admin moderation: removing fake or invalid issues and updating status
- User dashboard to track personal reports and impact

Out of scope (current version):

- Native mobile applications
- Automatic forwarding of issues to government departments
- Payment or donation features
- Offline reporting

1.3 Definitions, Acronyms and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
Civic Issue	A public problem in a community, such as damaged roads, garbage, water or electricity problems
User	A registered citizen who reports and follows issues
Admin	A privileged user who moderates issues and updates status
Kinde	Third-party authentication service used for login and sessions
Supabase	Backend platform providing the database and storage
Cloudinary	Cloud service used for image upload and delivery
UI	User Interface
FR / NFR	Functional Requirement / Non-Functional Requirement

1.4 References

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
- ResolIssue project README and source code repository

- Next.js, Tailwind CSS, Supabase, Kinde and Cloudinary official documentation

1.5 Document Overview

Section 2 gives the overall description of the product. Section 3 lists the functional requirements. Section 4 covers external interfaces. Section 5 lists non-functional requirements. Section 6 describes the system architecture and components. Section 7 describes the data requirements. Section 8 lists use cases and the issue lifecycle. Section 9 lists assumptions, risks and future enhancements.

2. Overall Description

2.1 Product Perspective

ResolIssue is a new, self-contained web application. It uses a component-based front end built with Next.js, with separate layers for the UI, authentication (Kinde) and data (Supabase). Images are handled by Cloudinary. The system is deployed on a cloud platform and accessed through a web browser.

2.2 Product Functions

- Register and authenticate users securely
- Report civic issues with description, images and exact location
- Browse issues and view details
- Like and comment on issues to show community priority
- Track issue progress through Pending, In-Progress and Resolved states
- Allow admins to update status and remove fake or invalid issues
- Show personal dashboards with contribution and impact

2.3 User Classes and Characteristics

User Class	Description	Permissions
Normal User	Citizen with basic web skills who wants to report and follow local issues	Report issues, like, comment, track own reports, view dashboard
Admin	Authorised person or authority member who manages the platform	All user permissions, update issue status, remove fake or invalid issues
Visitor (not logged in)	Person browsing without an account	View public issues only; must sign up or log in to take actions

2.4 Operating Environment

- Client: modern web browsers (Chrome, Edge, Brave, Firefox, Safari) on desktop, tablet and mobile
- Server: cloud-hosted Next.js application (Vercel) with Supabase backend services
- Network: active internet connection is required

2.5 Design and Implementation Constraints

- Front end must use Next.js (App Router) and Tailwind CSS
- Database and file storage must use Supabase
- Authentication must be handled through Kinde
- Images must be uploaded and served through Cloudinary
- Admin-only operations must be protected by role checks

2.6 Assumptions and Dependencies

- Users allow the browser to access location, or can mark a location manually
- Third-party services (Kinde, Supabase, Cloudinary) are available and within their usage limits
- Admin accounts are assigned by the project owner

3. Functional Requirements

Priority: High = must have, Medium = should have, Low = nice to have.

ID	Requirement	Priority
3.1 Authentication and User Management		
FR-01	The system shall allow new users to sign up and existing users to log in using Kinde authentication.	High
FR-02	The system shall maintain secure sessions and display the logged-in or logged-out state in the navigation bar.	High
FR-03	The system shall assign each account a role, either Normal User or Admin.	High
FR-04	The system shall allow only authenticated users to report issues, like and comment.	High
FR-05	The system shall allow users to log out, ending the session and blocking access to protected pages.	High
3.2 Issue Reporting		
FR-06	The system shall allow a logged-in user to create an issue with a title and description.	High
FR-07	The system shall allow users to attach one or more images, which are stored through Cloudinary.	High
FR-08	The system shall capture the precise location of the issue, using device location or manual selection.	High
FR-09	The system shall validate mandatory fields and file types before submission and show clear error messages.	High
FR-10	The system shall set the status of every new issue to Pending.	High
3.3 Viewing Issues		
FR-11	The system shall display issues as cards showing title, summary, image, location, status, like count and comment count.	High
FR-12	The system shall allow users to open an issue to view full details and its discussion.	High
FR-13	The system shall allow filtering or sorting of issues by status and by popularity (likes).	Medium
3.4 Community Engagement		
FR-14	The system shall allow a logged-in user to like (upvote) an issue, and to remove the like again.	High
FR-15	The system shall allow only one like per user for each issue.	High
FR-16	The system shall update the like count immediately after a like or unlike action.	High
FR-17	The system shall allow a logged-in user to post comments on an issue.	High
FR-18	The system shall display comments of an issue as a discussion thread in time order, with the commenter name.	Medium
3.5 Status Tracking		
FR-19	The system shall support the issue states Pending, In-Progress and Resolved.	High

ID	Requirement	Priority
FR-20	The system shall allow only Admin users to change the status of an issue.	High
FR-21	The system shall show the latest status to all users after a status change.	High
3.6 Admin Moderation		
FR-22	The system shall allow Admin users to remove fake, duplicate or invalid issues.	High
FR-23	The system shall allow Admin users to remove inappropriate comments.	Medium
FR-24	The system shall verify the Admin role on the server before performing any admin operation.	High
3.7 Dashboard		
FR-25	The system shall provide each user a dashboard listing the issues they reported with current status.	High
FR-26	The system shall show contribution and impact figures such as issues reported, issues resolved and likes received.	Medium

4. External Interface Requirements

4.1 User Interfaces

- Responsive layout designed with Tailwind CSS that works on mobile, tablet and desktop screens
- Consistent navigation bar with links, login/logout and user session information
- Issue form, issue cards, comment section, admin status control and dashboard screens
- Icons provided by Lucide Icons; clear feedback messages for success and errors

4.2 Hardware Interfaces

No special hardware is needed. The user device should have a web browser and, for accurate location, a GPS or location service. A camera or file storage is needed for adding images.

4.3 Software Interfaces

Interface	Purpose	Data Exchanged
Supabase	Database and storage for issues, comments, likes and users	Issue records, comments, likes, status values
Kinde Authentication	Sign up, login, session handling and user identity	User identity, session tokens, roles
Cloudinary	Image upload, optimisation and delivery	Image files and image URLs
Browser Geolocation / Map	Capture precise location of an issue	Latitude, longitude, address

4.3.1 Communication Interfaces

All communication between the browser, the application and third-party services shall use HTTPS.

5. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Pages shall load within 3 seconds on a normal broadband connection, and issue lists shall be fetched within 2 seconds.
NFR-02	Performance	Like and comment actions shall reflect on the screen within 1 second under normal load.
NFR-03	Security	All sensitive actions shall require authentication, and admin actions shall be checked on the server.
NFR-04	Security	All user input and uploaded files shall be validated for type and size, and protected against injection and script attacks.
NFR-05	Security	Sessions shall be invalidated on logout and protected pages shall not be accessible afterwards.
NFR-06	Usability	A new user shall be able to report an issue within 3 minutes without training.
NFR-07	Usability	The interface shall be fully responsive and work on screen widths from 320 px upwards.
NFR-08	Accessibility	Images shall have alternative text, and colour contrast and keyboard navigation shall follow basic accessibility guidelines.
NFR-09	Reliability	The system shall target 99% availability and shall show friendly error messages instead of crashing on failures.
NFR-10	Scalability	The backend shall scale through Supabase and the stateless front end shall scale horizontally with traffic.
NFR-11	Maintainability	The code shall use reusable components and a clean separation of UI, authentication and data layers.
NFR-12	Portability	The application shall work in the latest versions of Chrome, Edge, Brave, Firefox and Safari.

6. System Architecture

6.1 Architecture Overview

The system follows a layered, component-based architecture:

- **Presentation layer:** Next.js (App Router) pages and React components styled with Tailwind CSS
- **Authentication layer:** Kinde handles identity, login, sessions and role information
- **Data layer:** Supabase provides the centralised database with real-time reads and file storage
- **Media layer:** Cloudinary stores and delivers issue images

6.2 Key Components

Component	Responsibility
Navbar	Navigation links and session handling (login, logout, user info)
Signup	User onboarding and registration
IssueForm	Create a new issue with title, description, images and location
Issue	Issue card and summary view
Like	Upvote functionality with live like count
PostComment	Add a comment to an issue

Component	Responsibility
ShowComment	Render the discussion thread of an issue
UpdateStatus	Admin-only control to change the issue status

6.3 Technology Stack

Area	Technology
Framework	Next.js (App Router)
Styling	Tailwind CSS
Database and Storage	Supabase
Authentication	Kinde
Image Handling	Cloudinary
Icons	Lucide Icons
Deployment	Vercel (cloud hosting)

7. Data Requirements

The following logical data model is proposed. Exact table and column names may differ in the implementation.

Entity	Main Attributes
User	user_id (from Kinde), name, email, role (user or admin), created_at
Issue	issue_id, title, description, image_urls, latitude, longitude, address, status (pending, in-progress, resolved), reported_by (user_id), created_at, updated_at
Comment	comment_id, issue_id, user_id, content, created_at
Like	like_id, issue_id, user_id, created_at (unique for each issue and user pair)

Relationships: one User reports many Issues; one Issue has many Comments and many Likes; one User can write many Comments and give many Likes.

8. Use Cases and Issue Lifecycle

8.1 Use Case Summary

ID	Use Case	Actor	Description
UC-01	Sign up / Log in	User, Admin	User creates an account or logs in through Kinde and a session starts.
UC-02	Report Issue	User	User fills the issue form with title, description, images and location and submits it. Issue is saved as Pending.
UC-03	Like Issue	User	User upvotes an issue to show it is important. Like count increases.
UC-04	Comment on Issue	User	User adds a comment and it appears in the discussion thread.
UC-05	Track My Reports	User	User opens the dashboard to see own issues, their status and impact.

ID	Use Case	Actor	Description
UC-06	Update Status	Admin	Admin changes an issue to In-Progress or Resolved.
UC-07	Remove Issue	Admin	Admin deletes a fake or invalid issue from the platform.
UC-08	Log out	User, Admin	User ends the session and is redirected to the login page.

8.2 Issue Status Lifecycle

State	Meaning	Changed By
Pending	Issue is newly reported and waiting for review (default state).	System (on creation)
In-Progress	Issue has been accepted and work to fix it has started.	Admin
Resolved	The issue has been fixed and closed.	Admin

Flow: Pending, then In-Progress, then Resolved.

9. Assumptions, Risks and Future Enhancements

9.1 Risks

Risk	Mitigation
Fake or spam issues reduce trust in the platform	Admin moderation and removal of invalid issues; login required to report
Dependence on third-party services (Kinde, Supabase, Cloudinary)	Use official SDKs, monitor usage limits and handle service errors gracefully
Inaccurate location data from devices	Allow the user to adjust the pin or address manually before submitting
Unauthorised access to admin functions	Role checks on the server and restricted admin accounts

9.2 Future Enhancements

- Email or push notifications when the status of an issue changes
- Map view showing all issues in an area
- Categories and automatic detection of duplicate issues
- Assignment of issues to departments and resolution time tracking
- Mobile application and multi-language support